

Fundamentos del Software

UNIVERSIDAD ESTATAL DE MILAGRO
FACULTAD DE CIENCIAS E INGENIERÍA
CARRERA DE TECNOLOGÍAS DE LA
INFORMACIÓN EN LÍNEA

TEMA:

Fundamentos del Software

AUTOR:

Ángel Fernando Calero Maldonado
Ángel Fernando Calero Maldonado
Ángel Fernando Calero Maldonado
Ángel Fernando Calero Maldonado
Ángel Fernando Calero Maldonado

ASIGNATURA:

Fundamentos de tecnologías de información

DOCENTE:

Díaz Sandoya Eduardo Luis

FECHA DE ENTREGA:

miércoles, 21 de mayo de 2026

PERIODO:

abril 2026 a julio 2026

MILAGRO – ECUADOR

Índice

1	Introducción	3
2	Software: Clasificación	3
2.1	Clasificación según su función	3
2.2	Clasificación según tipo de licencia	4
3	Calidad del Software	5
3.1	La norma ISO/IEC 25010 y sus atributos	5
3.2	Evaluación de calidad: WhatsApp	6
4	Lenguajes de Alto y Bajo Nivel	7
4.1	Definición y diferencias fundamentales	7
4.2	Ventajas y desventajas	7
4.3	Ejemplo comparativo	7
5	Conclusiones	9
6	Referencias	10

1. Introducción

El software es uno de los componentes fundamentales de cualquier sistema informático moderno. Sin él, el hardware no pasaría de ser un conjunto de circuitos sin propósito definido. Comprender qué es el software, cómo se clasifica, con qué criterios se evalúa su calidad y de qué manera se construye mediante lenguajes de programación resulta esencial para cualquier profesional en formación dentro del área de Tecnologías de la Información.

Este trabajo aborda tres ejes temáticos establecidos en la guía de práctica G001_S06_TI_ENLÍNEA: la clasificación del software según su función y tipo de licencia, la evaluación de calidad bajo la norma ISO/IEC 25010 y el análisis comparativo entre lenguajes de alto y bajo nivel. El objetivo es ir más allá de las definiciones formales y conectar cada concepto con ejemplos reales del entorno tecnológico actual.

2. Software: Clasificación

2.1 Clasificación según su función

El software puede agruparse en tres grandes categorías según la función que desempeña dentro de un sistema informático: software de sistema, software de aplicación y software de programación. Cada categoría cumple un rol distinto y se dirige a usuarios o procesos específicos.

El **software de sistema** es el conjunto de programas que permiten que el hardware funcione y que sirven como plataforma para ejecutar otros programas. Gestiona los recursos del computador y ofrece servicios básicos al resto del software. El **software de aplicación** está diseñado para que el usuario realice tareas concretas, como redactar documentos, comunicarse o gestionar datos. Por su parte, el **software de programación** proporciona las herramientas necesarias para que los desarrolladores creen nuevos programas, incluyendo compiladores, intérpretes y entornos de desarrollo integrados (Pressman, 2014).

Categoría	Ejemplo	Descripción breve
Software de sistema	Windows 11	Sistema operativo de Microsoft para equipos personales y empresariales.
	Linux Ubuntu	Sistema operativo libre orientado a servidores y desarrollo.
	macOS Sonoma	Sistema operativo de Apple optimizado para su ecosistema de hardware.
Software de aplicación	Microsoft Word	Procesador de texto para la creación y edición de documentos.
	WhatsApp	Aplicación de mensajería instantánea y llamadas de voz y video.
	Google Drive	Almacenamiento y edición colaborativa de archivos en la nube.
Software de programación	Visual Studio Code	Editor de código con soporte para múltiples lenguajes y extensiones.
	IntelliJ IDEA	IDE especializado en Java con herramientas avanzadas de refactorización.
	Python (intérprete)	Entorno de ejecución e interpretación del lenguaje Python.

Tabla 1. Clasificación del software según su función con ejemplos representativos.

2.2 Clasificación según tipo de licencia

Además de su función, el software también puede clasificarse según las condiciones legales que regulan su uso, distribución y modificación. Esta distinción tiene implicaciones directas en el acceso a la tecnología, los costos de implementación y la posibilidad de adaptar el software a necesidades específicas (Stallman, 2002).

Tipo de licencia	Ejemplo	Descripción breve
Software libre	GNU/Linux	Núcleo del sistema operativo Linux, distribuido bajo licencia GPL.
	LibreOffice	Suite ofimática libre como alternativa a Microsoft Office.
	VLC Media Player	Reproductor multimedia libre compatible con múltiples formatos.
Software propietario	Adobe Photoshop	Editor gráfico profesional con licencia comercial de Adobe.
	Microsoft Office	Suite de productividad con licencia de pago o suscripción.
	AutoCAD	Software de diseño asistido por computadora con licencia anual.
Código abierto	Apache HTTP Server	Servidor web de código abierto ampliamente usado en producción.
	MySQL	Sistema de gestión de bases de datos relacional de código abierto.
	Firefox	Navegador web de código abierto desarrollado por Mozilla.
Software en la nube	Google Docs	Editor de texto en línea sin instalación local requerida.
	Canva	Plataforma de diseño gráfico accesible desde el navegador.
	Zoom	Plataforma de videoconferencias y reuniones virtuales en la nube.

Tabla 2. Clasificación del software según tipo de licencia con ejemplos.

3. Calidad del Software

3.1 La norma ISO/IEC 25010 y sus atributos

La norma ISO/IEC 25010 es el estándar internacional que define un modelo de calidad del producto software. Este modelo establece ocho características principales que permiten evaluar de manera sistemática qué tan bueno es un software desde múltiples perspectivas: funcionalidad, fiabilidad, usabilidad, eficiencia de desempeño, mantenibilidad, portabilidad, seguridad y compatibilidad (ISO/IEC 25010, 2011).

La **funcionalidad** mide si el software cumple con las funciones para las que fue diseñado. La **fiabilidad** evalúa la capacidad del sistema para mantener un nivel de servicio bajo condiciones determinadas. La **usabilidad** se refiere a la facilidad con que los usuarios pueden aprender a operar y controlar el sistema. La **eficiencia de desempeño** analiza el uso óptimo de

recursos como tiempo de respuesta y consumo de memoria. La **mantenibilidad** mide con qué facilidad puede modificarse el software. La **portabilidad** evalúa la capacidad de transferir el software a distintos entornos. La **seguridad** considera la protección de datos y la resistencia ante accesos no autorizados. Finalmente, la **compatibilidad** examina la coexistencia del sistema con otros productos en el mismo entorno.

3.2 Evaluación de calidad: WhatsApp

Se seleccionó WhatsApp como software de análisis por ser una de las aplicaciones de comunicación más utilizadas a nivel mundial, lo que permite contar con suficiente información empírica para una evaluación fundamentada.

Atributo ISO/IEC 25010	Fortaleza	Debilidad
Funcionalidad	Mensajería, llamadas, video-llamadas, archivos y estados en una sola app.	Limitaciones para gestionar mensajes en múltiples dispositivos simultáneamente.
Fiabilidad	Alta disponibilidad global con mínimo tiempo de inactividad.	Interrupciones del servicio afectan a millones de usuarios de forma simultánea.
Usabilidad	Interfaz intuitiva que no requiere capacitación previa para uso básico.	Configuraciones avanzadas de privacidad poco visibles para usuarios no técnicos.
Eficiencia de desempeño	Bajo consumo de datos en mensajes de texto y audio comprimido.	Llamadas de video en redes lentas presentan latencia y pérdida de calidad notable.
Mantenibilidad	Actualizaciones frecuentes con mejoras progresivas y corrección de errores.	El código fuente es propietario; no permite auditorías externas independientes.
Portabilidad	Disponible en Android, iOS y versión web/desktop multi-plataforma.	La versión de escritorio depende de la conexión activa del teléfono móvil.
Seguridad	Cifrado de extremo a extremo activado por defecto en todos los chats.	Los metadatos de uso son recopilados por Meta con fines comerciales.
Compatibilidad	Se integra con el sistema de contactos del dispositivo sin configuración adicional.	Compatibilidad limitada con soluciones empresariales de gestión de comunicaciones.

Tabla 3. Evaluación de calidad de WhatsApp bajo los atributos de la norma ISO/IEC 25010.

4. Lenguajes de Alto y Bajo Nivel

4.1 Definición y diferencias fundamentales

Un **lenguaje de alto nivel** es aquel cuya sintaxis está diseñada para que los seres humanos puedan leerlo y escribirlo con relativa facilidad. Utiliza palabras y estructuras cercanas al lenguaje natural, abstrayendo los detalles del hardware subyacente. Python, Java, C# y JavaScript son ejemplos representativos. El programador no necesita preocuparse por cómo se gestiona la memoria o cómo se ejecutan internamente las instrucciones; el compilador o intérprete se encarga de traducir el código al lenguaje que entiende la máquina.

Un **lenguaje de bajo nivel**, en cambio, opera cerca del hardware y requiere que el programador tenga un conocimiento detallado de la arquitectura del procesador. El ensamblador (Assembly) es el ejemplo más representativo: cada instrucción corresponde casi directamente a una operación del procesador. El lenguaje máquina, compuesto únicamente de ceros y unos, es el nivel más bajo posible (Tanenbaum, 2016).

4.2 Ventajas y desventajas

Los lenguajes de alto nivel ofrecen como principales ventajas la legibilidad del código, la rapidez de desarrollo, la portabilidad entre diferentes plataformas y una curva de aprendizaje más accesible. Sin embargo, tienen como desventaja que el programador tiene menor control sobre los recursos del sistema y que la ejecución puede ser menos eficiente que en bajo nivel.

Los lenguajes de bajo nivel permiten un control preciso sobre el hardware, optimización del rendimiento al máximo y son indispensables en sistemas embebidos, controladores de dispositivos y desarrollo de sistemas operativos. Su principal desventaja es que son difíciles de leer, escribir y mantener, y están fuertemente ligados a una arquitectura específica.

4.3 Ejemplo comparativo

A continuación se presenta una comparación entre un programa que imprime un mensaje en pantalla, escrito en Python (alto nivel) y su equivalente conceptual en lenguaje ensamblador x86-64 para Linux (bajo nivel).

Python (alto nivel):

```
print("Hola, mundo")
```

Con una sola línea de código, Python gestiona internamente la asignación de memoria para la cadena, la llamada al sistema operativo y la presentación del resultado en la salida estándar.

Ensamblador x86-64 (bajo nivel):

```
section .data
    msg db "Hola, mundo", 10
    len equ $ - msg
section .text
    global _start
_start:
    mov rax, 1
    mov rdi, 1
    mov rsi, msg
    mov rdx, len
    syscall
    mov rax, 60
    xor rdi, rdi
    syscall
```

En ensamblador, el programador debe definir manualmente el segmento de datos, declarar el mensaje como una secuencia de bytes, especificar el número de la llamada al sistema (syscall) para escritura en pantalla, cargar los registros del procesador con los parámetros correspondientes y finalmente invocar la instrucción de salida. Lo que Python resuelve en una línea, en ensamblador requiere más de una docena de instrucciones explícitas.

Característica	Alto nivel (Python)	Bajo nivel (Ensamblador)
Legibilidad	Muy alta; similar al lenguaje natural.	Muy baja; orientada al procesador.
Velocidad de desarrollo	Rápida; pocas líneas de código.	Lenta; instrucciones muy detalladas.
Control del hardware	Limitado; lo gestiona el intérprete.	Total; el programador controla registros y memoria.
Portabilidad	Alta; ejecutable en múltiples sistemas.	Baja; dependiente de la arquitectura del CPU.
Rendimiento	Aceptable para la mayoría de tareas.	Máximo; ideal para sistemas críticos.
Uso principal	Desarrollo web, IA, scripting, ciencia de datos.	Sistemas embebidos, firmware, drivers.

Tabla 4. Comparativa entre lenguajes de alto nivel y bajo nivel.

5. Conclusiones

La clasificación del software en categorías funcionales y por tipo de licencia no es un ejercicio meramente teórico: tiene consecuencias directas en las decisiones tecnológicas que toman las organizaciones, desde la elección de herramientas de desarrollo hasta la estrategia de implementación de sistemas. Conocer estas categorías permite evaluar opciones con criterio técnico y económico.

La norma ISO/IEC 25010 ofrece un marco riguroso y estandarizado para evaluar la calidad del software más allá de si «funciona» o no. El análisis de WhatsApp demuestra que incluso los productos más populares presentan debilidades significativas en dimensiones como seguridad de metadatos o dependencia de conectividad. Esta perspectiva crítica es fundamental para el ejercicio profesional en TI.

La comparación entre lenguajes de alto y bajo nivel ilustra que no existe una opción universalmente superior: cada nivel responde a necesidades distintas. Python permite desarrollar soluciones rápidas y portables, mientras que el ensamblador es insustituible cuando se requiere máximo control sobre el hardware. Un ingeniero en TI competente debe comprender ambos extremos del espectro para tomar decisiones de diseño fundamentadas.

6. Referencias

- ISO/IEC 25010. (2011). *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. International Organization for Standardization.
- Pressman, R. S. (2014). *Ingeniería del software: Un enfoque práctico* (8.a ed.). McGraw-Hill Education.
- Stallman, R. M. (2002). *Free software, free society: Selected essays of Richard M. Stallman*. GNU Press.
- Tanenbaum, A. S. (2016). *Structured computer organization* (6.a ed.). Pearson.
- WhatsApp LLC. (2023). *WhatsApp privacy policy*. Meta Platforms. <https://www.whatsapp.com/legal/privacy-policy>